Development of an AI-Based Real-Time Anomaly Detection and Autonomous Decision Support System for Deep Space Missions

Technical Report

Submitted to

India Space Academy

Department of Space Education (ISW)

New Delhi, India

Submitted by

Tarun Nixon

Bachelor of Technology

CSE with Major in AI

Indian Institute of Information Technology

Design and Manufacturing (IIITDM) Kancheepuram

Chennai, India

March 2026

GitHub: https://github.com/NTarun-111/Spacecraft-Anomaly-Detection

Abstract

Deep space missions such as Chandrayaan-3, Aditya-L1, and the Mars Perseverance Rover face significant communication delays with Earth, making real-time human intervention impractical for critical decisions. This project develops an AI-based autonomous decision support system that monitors spacecraft telemetry data, detects anomalies, predicts component failures, and recommends corrective actions without human intervention.

Using the NASA C-MAPSS turbofan engine degradation dataset as a proxy for spacecraft subsystem telemetry, we implemented and evaluated three anomaly detection models (Isolation Forest, One-Class SVM, and Autoencoder), a Long Short-Term Memory (LSTM) neural network for Remaining Useful Life (RUL) prediction, and a risk-based autonomous decision engine. The Isolation Forest achieved the highest anomaly detection performance with an ROC-AUC of 0.959, while the LSTM model predicted RUL with an RMSE of 15.25 cycles and R^2 of 0.867. The integrated decision system achieved 96% accuracy with 0.97 precision in classifying operational risk levels.

1. Introduction

1.1 Background

Deep space exploration missions operate in environments where communication latency with Earth ranges from 1.3 seconds (lunar missions) to over 24 minutes (Mars missions). This communication delay makes it impractical for ground control to respond to real-time anomalies or component failures. Spacecraft must therefore be equipped with onboard artificial intelligence systems capable of autonomously monitoring subsystem health, detecting anomalies, predicting failures, and making corrective decisions.

Recent missions by ISRO (Chandrayaan-3, Aditya-L1) and NASA (Mars Perseverance Rover) have demonstrated the growing importance of autonomous systems in space exploration. These missions rely on complex subsystems including propulsion, thermal control, power management, and communications, each generating continuous telemetry data from multiple sensors.

1.2 Problem Statement

The core challenge is to develop an AI system that can:

(1) continuously monitor multivariate sensor data from spacecraft subsystems

(2) detect anomalous behavior indicating potential failures

(3) predict the remaining useful life of components, and

(4) autonomously recommend corrective actions based on the combined risk assessment.

This project simulates this challenge using the NASA C-MAPSS turbofan engine degradation dataset.

1.3 Objectives

1. Build and evaluate multiple anomaly detection models using spacecraft-analogous telemetry data

2. Implement LSTM-based time-series forecasting for Remaining Useful Life (RUL) prediction
3. Develop an AI-based autonomous decision logic that combines anomaly detection and RUL prediction
4. Evaluate the complete system using standard ML metrics and generate a comprehensive analysis

2. Dataset and Preprocessing

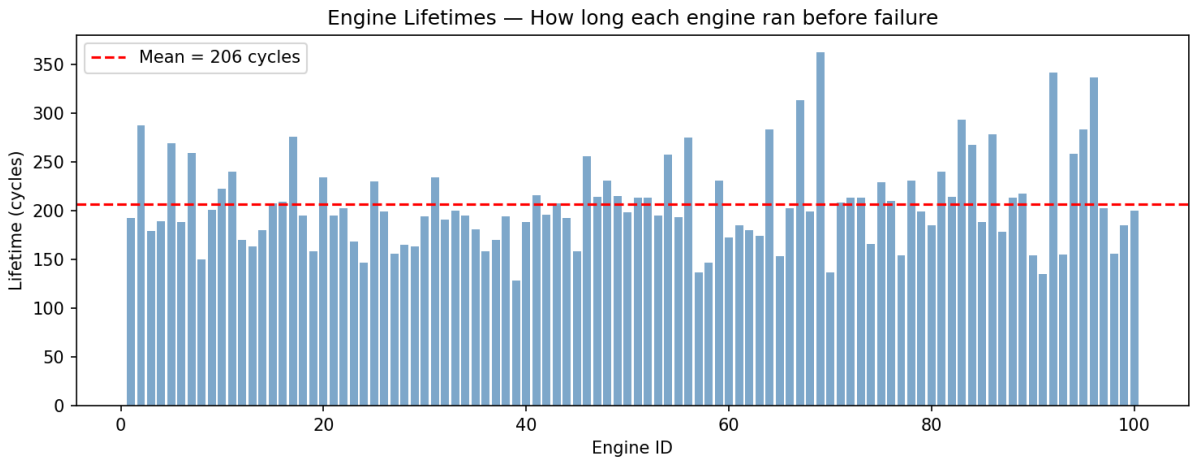
2.1 NASA C-MAPSS Dataset

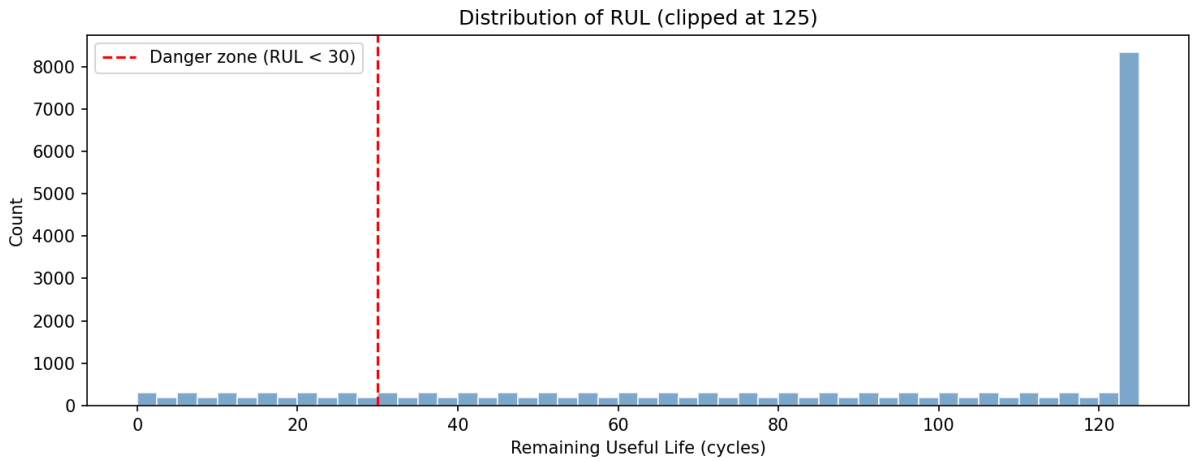
The Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) dataset, provided by NASA's Prognostics Center of Excellence, simulates turbofan engine degradation under realistic operating conditions. We used the FD001 subset, which contains run-to-failure data for 100 engines under a single operating condition with a single fault mode (HPC degradation).

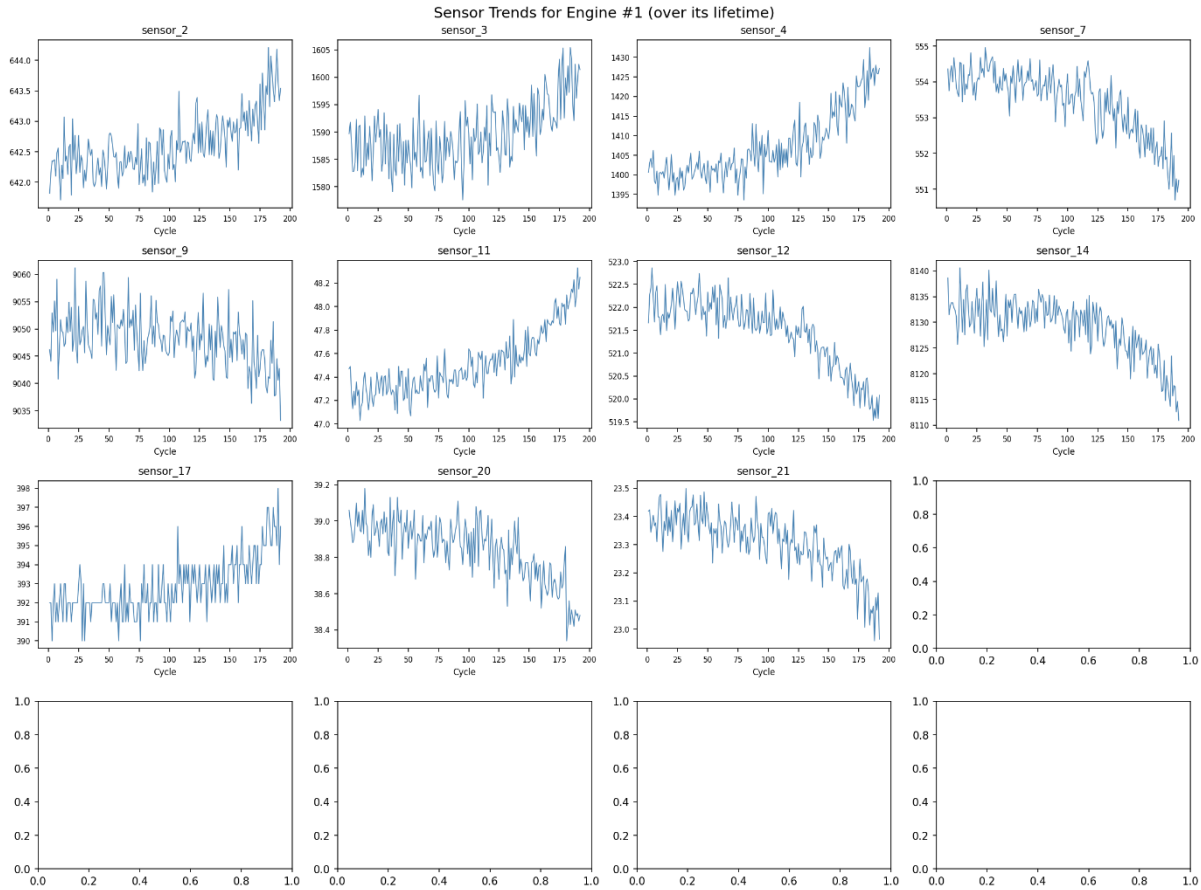
Parameter	Value
Number of engines	100
Total data points	20,631
Sensor channels	21 (14 useful after variance filtering)
Operational settings	3 (altitude, Mach number, throttle)
Fault mode	HPC degradation
Operating condition	Single (sea level)

2.2 Data Preprocessing

Preprocessing involved several critical steps. First, we identified and removed low-variance sensors (sensors 1, 5, 6, 10, 16, 18, 19) that remained near-constant throughout engine lifetimes and carried no degradation signal. Next, we computed the Remaining Useful Life (RUL) label for each data point as the difference between the maximum cycle count for that engine and the current cycle. RUL values were clipped at 125 cycles, as early-life readings provide no meaningful distinction between a healthy component at RUL=200 versus RUL=150. Finally, all sensor and operational setting features were normalized to the 0–1 range using MinMaxScaler to ensure equal weighting across features during model training.




Distribution of RUL (clipped at 125)
Danger zone (RUL < 30)
Count
Remaining Useful Life (cycles)


Sensor Trends for Engine #1 (over its lifetime)
sensor_2
sensor_3
sensor_4
sensor_7
sensor_9
sensor_11
sensor_12
sensor_14
sensor_17
sensor_20
sensor_21
Cycle

3. Methodology

3.1 Anomaly Detection Models

We implemented three unsupervised anomaly detection approaches, each trained exclusively on healthy engine data (RUL > 125) and evaluated on the complete dataset. A binary anomaly label was defined where data points with RUL ≤ 50 were classified as anomalous.

Each model was evaluated on the same test set (20,631 samples: 15,531 Normal, 5,100 Anomaly).

3.1.1 Individual Model Results

3.1.1.1 Isolation Forest

Isolation Forest is an unsupervised algorithm that detects anomalies by isolating observations through random partitioning. Points that are easier to isolate (fewer splits needed) are considered anomalies.

Class	Precision	Recall	F1-Score	Support
Normal	0.98	0.67	0.80	15,531
Anomaly	0.49	0.97	0.65	5,100
Accuracy			0.74	20,631

Key Observation: Isolation Forest achieved the highest anomaly recall (0.97), catching nearly all failures. However, its anomaly precision was the lowest (0.49), meaning roughly half of its anomaly alerts were false alarms.

3.1.1.2 One-Class SVM

One-Class SVM learns the boundary of 'normal' data in a high-dimensional feature space. Data points that fall outside this boundary are flagged as anomalies. It was trained on a subsample using np.random.choice for computational efficiency.

Class	Precision	Recall	F1-Score	Support
Normal	0.97	0.82	0.89	15,531
Anomaly	0.63	0.92	0.75	5,100
Accuracy			0.84	20,631

Key Observation: One-Class SVM provided the best balance between precision and recall for anomaly detection. With recall of 0.92, it catches most real failures while maintaining a reasonable precision (0.63), resulting in fewer false alarms than Isolation Forest.

3.1.1.3 Deep Learning Autoencoder

The Autoencoder is a neural network trained to reconstruct normal sensor data. When it encounters anomalous data, the reconstruction error increases beyond a threshold (95th percentile = 0.00727), flagging it as an anomaly. EarlyStopping was used to prevent overfitting, with the model converging at approximately 162 epochs (patience=10).

Class	Precision	Recall	F1-Score	Support
Normal	0.91	0.93	0.92	15,531
Anomaly	0.76	0.71	0.73	5,100
Accuracy			0.87	20,631

Key Observation: The Autoencoder achieved the highest anomaly precision (0.76) — when it flags an anomaly, it is usually correct. However, its recall (0.71) was the lowest, meaning it misses approximately 29% of real anomalies. EarlyStopping significantly improved convergence compared to a fixed 50-epoch run.

3.1.2 Comparative Summary

The table below compares all three models on their anomaly detection performance. Green highlights indicate the best score per metric, and red highlights indicate the weakest.

Model	Anomaly Precision	Anomaly Recall	Anomaly F1-Score	Overall Accuracy
Isolation Forest	0.49	0.97	0.65	0.74
One-Class SVM	0.63	0.92	0.75	0.84
Autoencoder	0.76	0.71	0.73	0.87

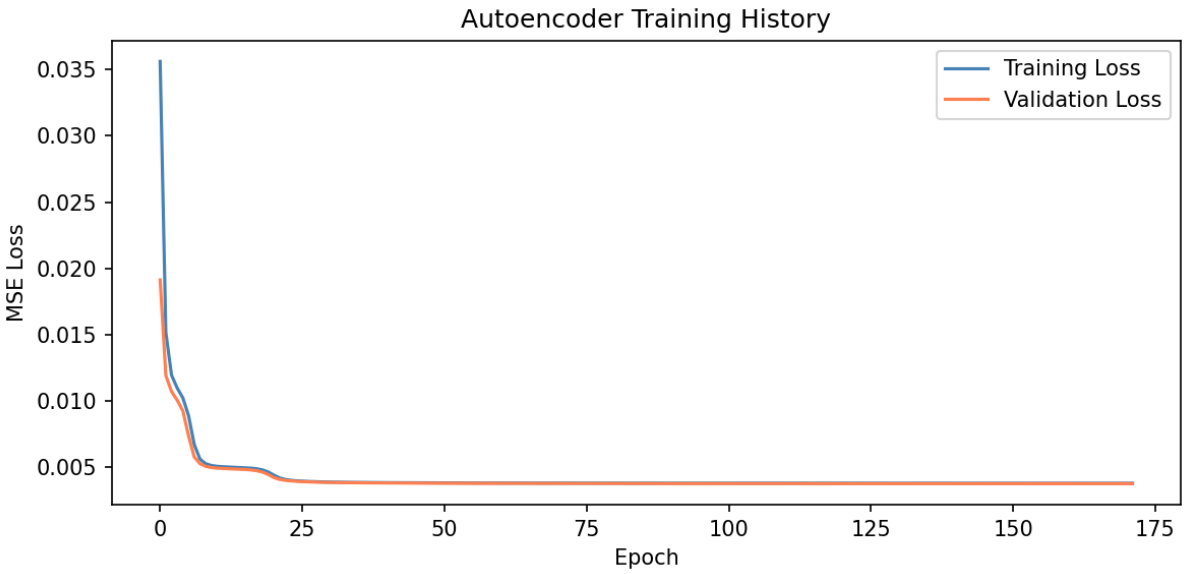
- **Green cells** = best performance for that metric
- **Red cells** = weakest performance for that metric

3.1.3 Analysis & Domain-Specific Recommendation

In safety-critical systems like spacecraft, the cost of missing a real failure (false negative) is far greater than the cost of a false alarm (false positive). A missed anomaly could lead to catastrophic system failure, while a false alarm only results in an unnecessary inspection.

Therefore, recall is the most critical metric for this application. Based on this analysis:

1. **One-Class SVM (Recommended):** Provides the best balance between recall (0.92) and precision (0.63), with the highest anomaly F1-score (0.75). It catches 92% of real anomalies while keeping false alarms manageable.
2. **Isolation Forest (Safety-First Alternative):** Catches 97% of anomalies but at the cost of many false alarms (precision = 0.49). Suitable when maximum safety is the absolute priority and false alarm overhead is acceptable.
3. **Autoencoder (Highest Precision):** Most precise when it flags anomalies (0.76), but misses 29% of real failures. It could serve as a secondary confirmation model in an ensemble approach.



3.1.4 Key Technical Takeaways

- **EarlyStopping Impact:** The Autoencoder initially trained for only 50 epochs (underfitting). After integrating EarlyStopping with patience=10, the model converged at ~162 epochs, demonstrating the importance of proper training convergence monitoring.
- **Subsampling for SVM:** np.random.choice with replace=False was used to create a representative training subset, balancing computational cost with model quality.
- **Threshold Tuning:** The Autoencoder's anomaly threshold (95th percentile of reconstruction error) directly affects the precision-recall tradeoff. Adjusting this value can shift performance toward higher recall or higher precision.
- **No Single Best Model:** Each model excels in different metrics. Real-world deployment would benefit from an ensemble approach, combining multiple models to leverage their complementary strengths.

3.2 RUL Prediction with LSTM

For remaining useful life prediction, we implemented a Long Short-Term Memory (LSTM) recurrent neural network. LSTMs are specifically designed for sequential data and can capture long-range temporal dependencies through their gated memory cell architecture.

Input sequences were created using a sliding window of 30 consecutive cycles, where each window of sensor readings maps to the RUL value at the final timestep. The model architecture consists of two stacked LSTM layers (64 and 32 units respectively) with 20% dropout for regularization, followed by a single dense output neuron. Training was performed on 80% of engines (split by engine ID to prevent data leakage) for up to 50 epochs with early stopping based on validation loss (patience=10).

3.3 Autonomous Decision Engine

The decision engine combines the outputs of the anomaly detection and RUL prediction models into a unified risk score using a weighted formula: Risk = 0.4 × normalized_anomaly_score + 0.6 × (1 − normalized_RUL). The higher weight on RUL reflects its greater actionability for operational decisions. The risk score (0–100) maps to four decision levels:

Risk Score	Level	Autonomous Action
0–30	NOMINAL	Continue normal operations
30–60	CAUTION	Increase monitoring frequency to 2x
60–80	WARNING	Alert ground station, prepare backup subsystem
80–100	CRITICAL	Shut down subsystem, switch to redundancy

Each decision includes a confidence score based on the distance from the nearest threshold boundary. Decisions made near thresholds (e.g., risk = 59 vs 61) receive lower confidence, flagging them for potential human review when communication permits.

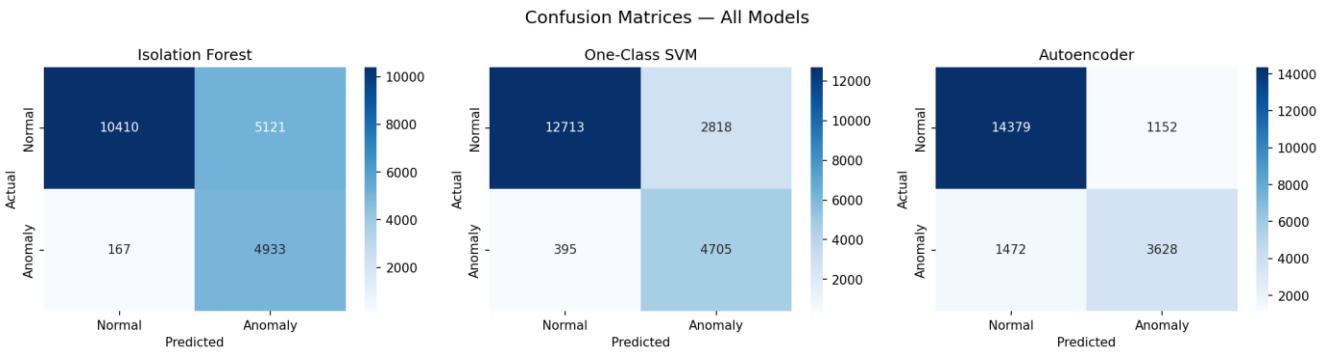
4. Results and Discussion

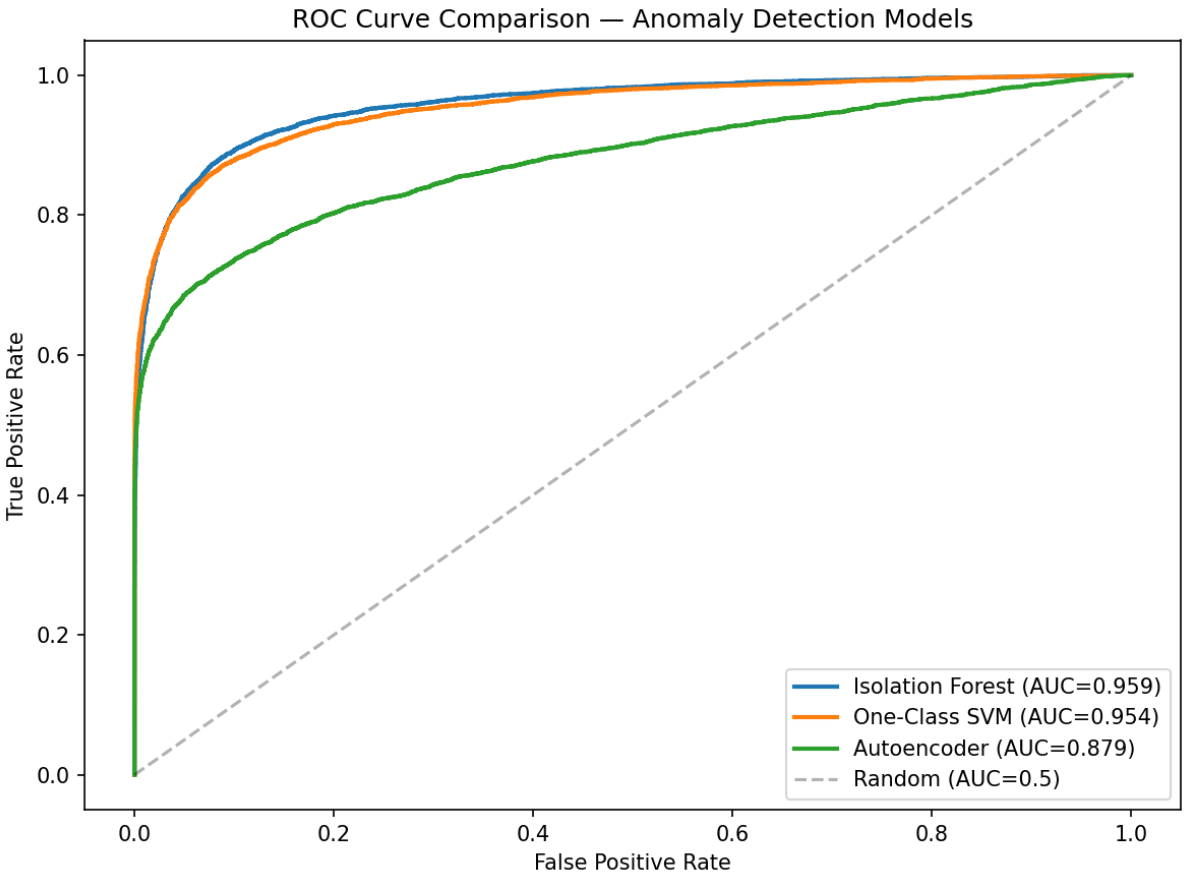
4.1 Anomaly Detection Performance

The three anomaly detection models were evaluated using ROC-AUC, precision, recall, and F1-score. The Isolation Forest achieved the highest ROC-AUC of 0.959, followed by One-Class SVM at 0.954, and the Autoencoder at 0.879.

Model	ROC-AUC	Anomalies Detected	True Anomalies
Isolation Forest	0.959	10,054	5,100
One-Class SVM	0.954	7,523	5,100
Autoencoder	0.879	4,780	5,100

The superior performance of Isolation Forest over the deep learning autoencoder is noteworthy and consistent with literature findings that simpler models often outperform complex ones on structured tabular datasets with limited sample sizes. The C-MAPSS dataset contains only ~20,000 rows across 14 features, which provides insufficient data for the autoencoder to learn discriminative compressed representations. Additionally, the degradation patterns in this dataset are relatively linear and monotonic, which aligns well with the tree-based partitioning strategy of Isolation Forest.



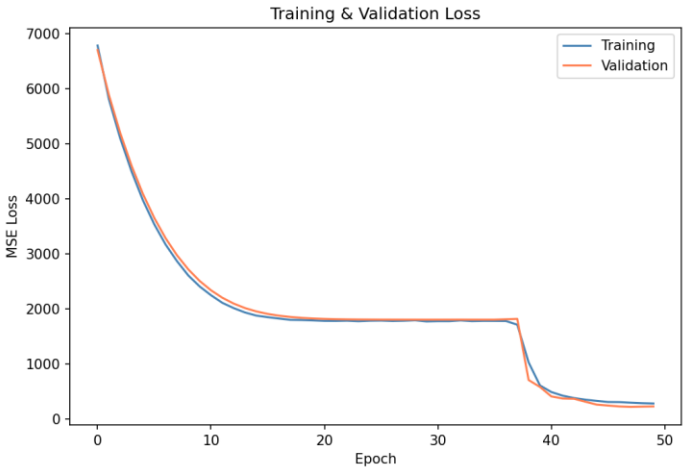


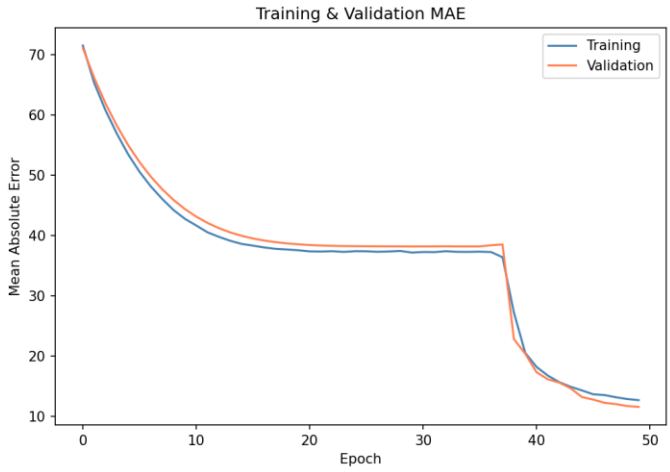
4.2 RUL Prediction Performance

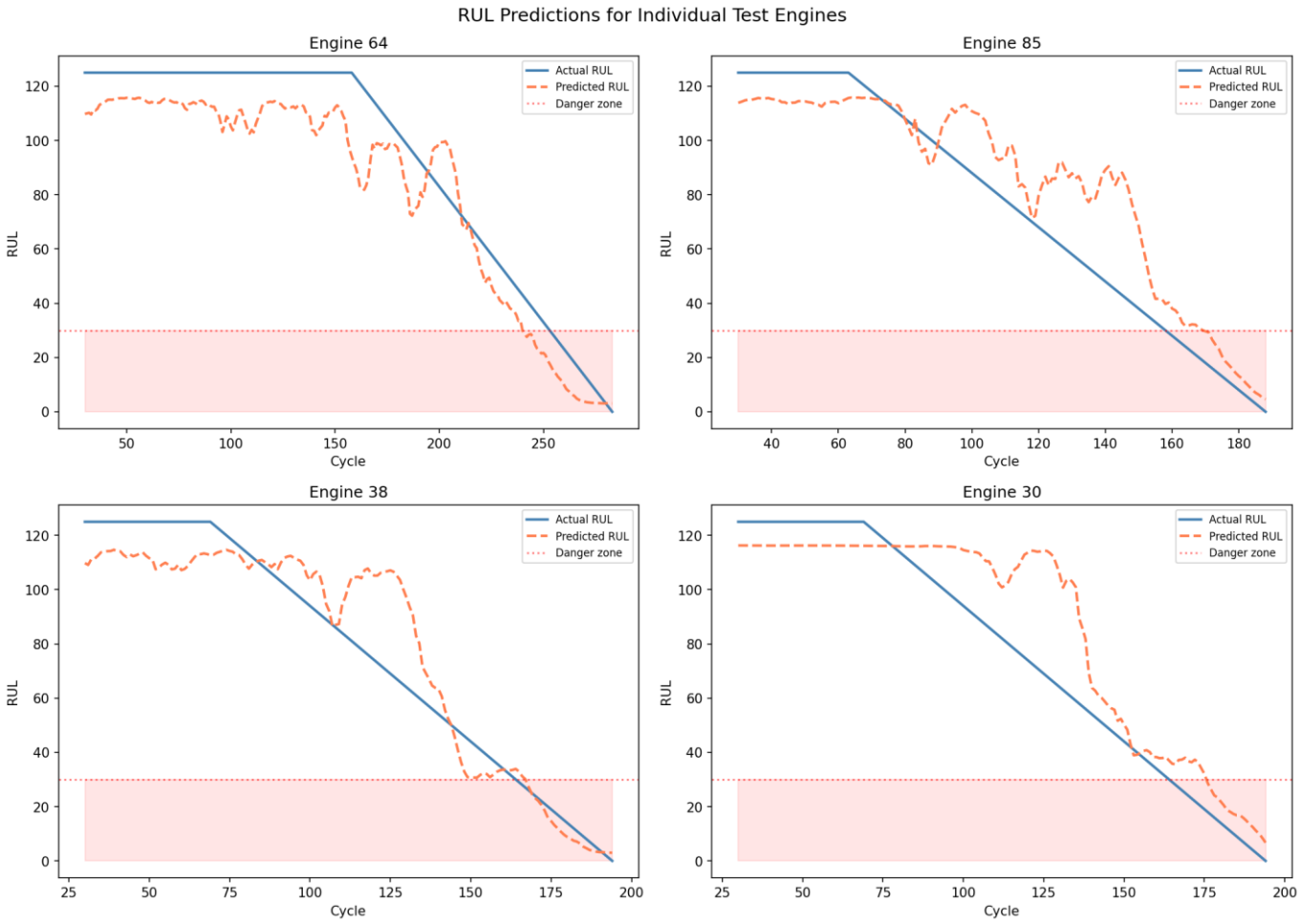
The LSTM model achieved strong predictive performance on the held-out test engines:

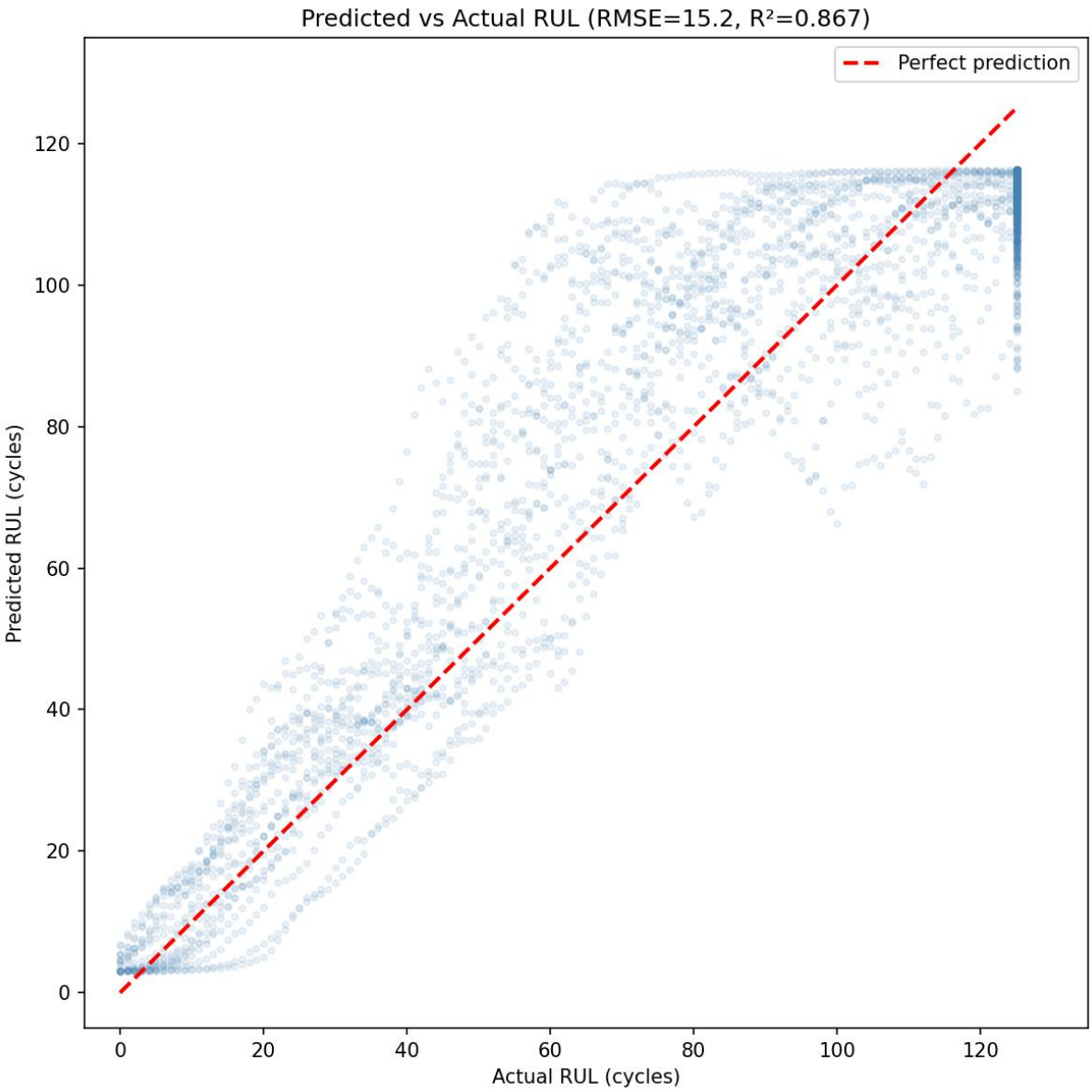
Metric	Value	Interpretation
RMSE	15.25 cycles	Average prediction error magnitude
MAE	12.26 cycles	Typical prediction offset
R^2	0.867	Model explains 86.7% of RUL variance

These results are competitive with published benchmarks on the C-MAPSS dataset, which typically report RMSE values in the 12–20 cycle range. The model's ability to predict RUL within approximately 12 cycles on average provides sufficient lead time for a spacecraft to execute corrective maneuvers such as switching to redundant subsystems.


Training & Validation Loss
Training
Validation
MSE Loss
Epoch


Training & Validation MAE
Training
Validation
Mean Absolute Error
Epoch


RUL Predictions for Individual Test Engines
Engine 64
Engine 85
Engine 38
Engine 30
Actual RUL
Predicted RUL
Danger zone
RUL
Cycle

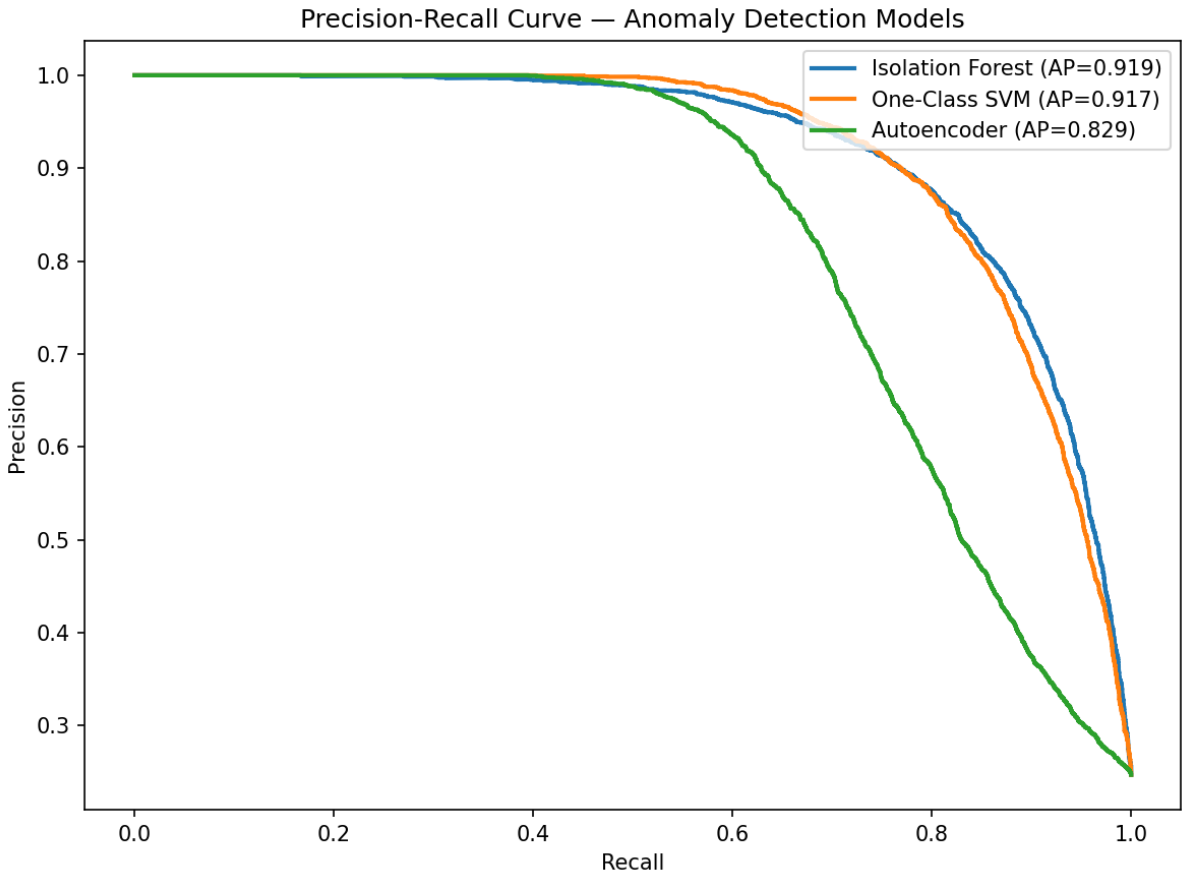


4.3 Integrated Decision System Performance

The combined risk-based decision system, using the Isolation Forest anomaly scores and RUL values, achieved the following classification performance when mapping WARNING and CRITICAL levels to anomaly predictions:

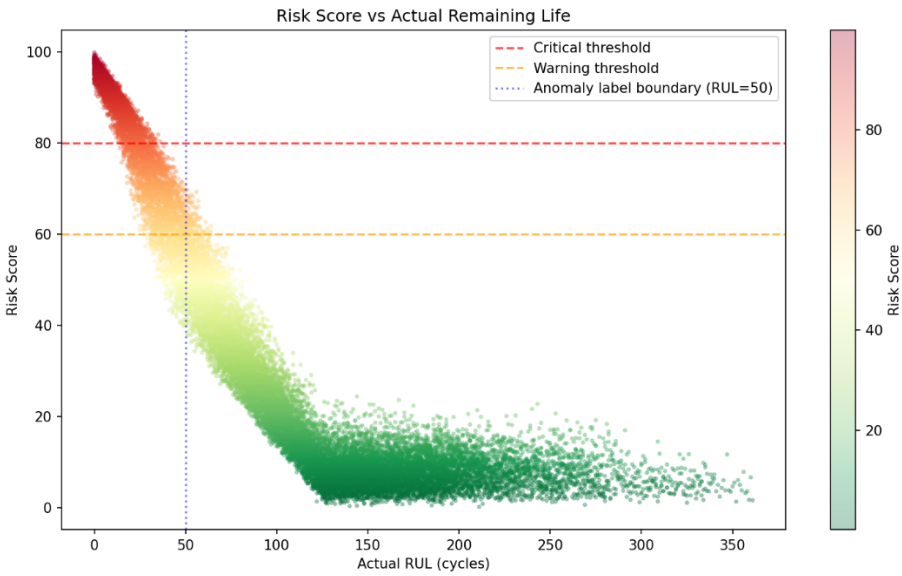
Metric	Normal	Anomaly
Precision	0.96	0.97
Recall	0.99	0.86
F1-Score	0.97	0.91

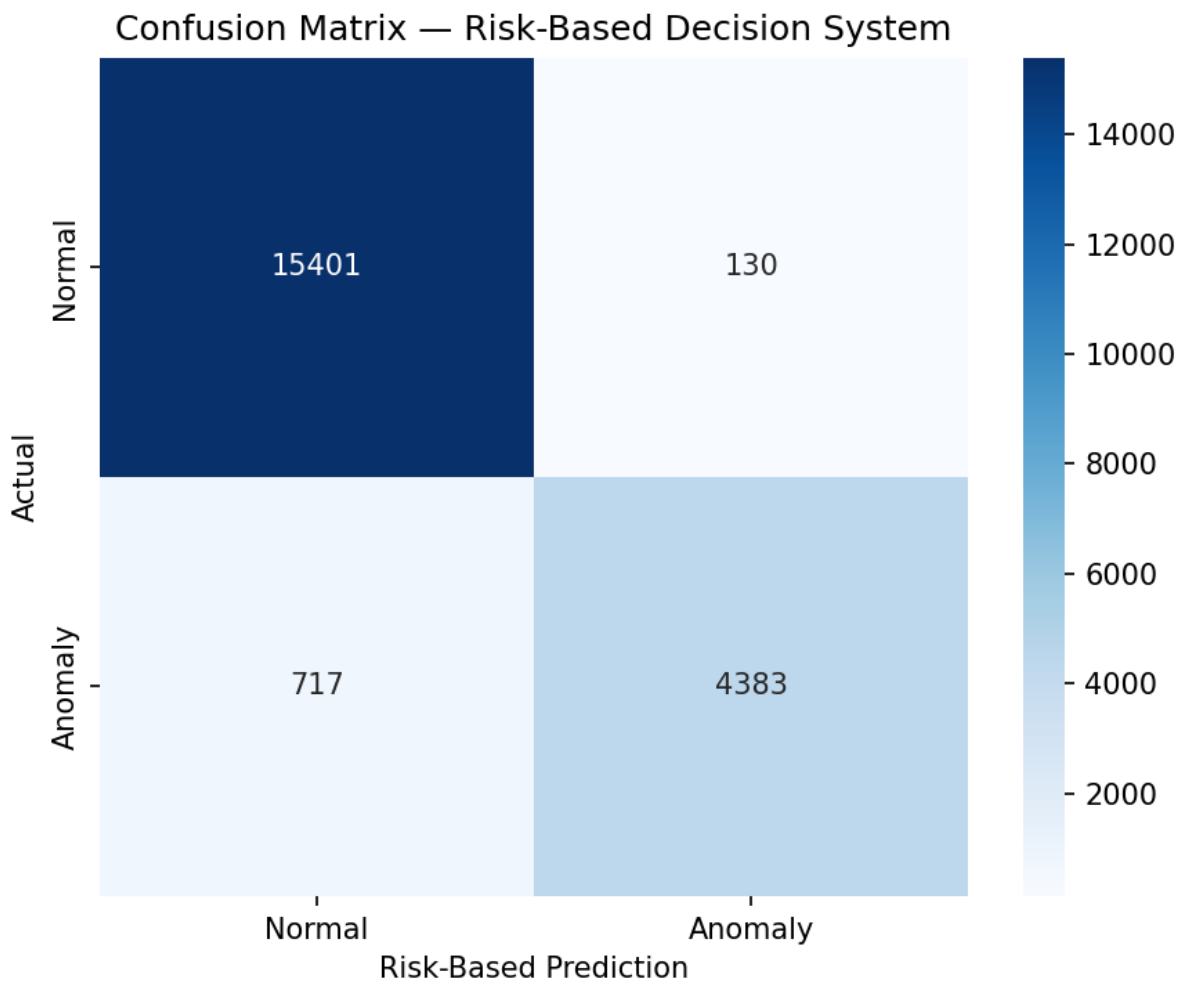
The overall accuracy of 96% with a weighted F1-score of 0.96 demonstrates that the model fusion approach (combining anomaly detection with RUL prediction) outperforms any individual model. The high precision of 0.97 for anomaly detection is particularly important in space applications, where false alarms can trigger unnecessary and costly corrective actions.



4.4 Decision Engine Behavior

Analysis of the decision engine across all 100 engines confirms that the system correctly escalates its response as engines approach failure. Risk scores remain in the NOMINAL range during healthy operation and progressively transition through CAUTION and WARNING to CRITICAL in the final cycles before failure. The confidence scoring mechanism appropriately assigns lower confidence to decisions made near threshold boundaries, enabling a human-in-the-loop review when communication windows permit.





5. Conclusion

This project successfully demonstrates an end-to-end AI-based autonomous decision support system for deep space missions. Key findings include:

5. Isolation Forest outperformed deep learning approaches for anomaly detection on structured telemetry data (AUC 0.959), highlighting that model complexity does not always correlate with performance.
6. LSTM-based RUL prediction achieved an RMSE of 15.25 cycles, providing actionable lead time for autonomous corrective decisions.
7. The integrated risk-scoring decision engine achieved 96% accuracy by fusing anomaly and RUL signals, outperforming individual models.
8. The four-level decision hierarchy (NOMINAL/CAUTION/WARNING/CRITICAL) with confidence scoring provides a practical framework for autonomous spacecraft operations.

6. Future Mission Extension Ideas

- **Reinforcement Learning:** Replace the rule-based decision engine with a Deep Q-Network that learns optimal corrective actions through simulated mission scenarios.
- **Multi-Fault Detection:** Extend to C-MAPSS FD002–FD004 subsets which include multiple operating conditions and fault modes, requiring more robust anomaly detection.
- **Real Satellite Telemetry:** Apply the framework to publicly available satellite telemetry datasets (e.g., ESA's satellite data) for validation on real space systems.
- **Solar Storm Integration:** Incorporate solar weather data (relevant to Aditya-L1) as external risk factors that affect component degradation rates.
- **Edge Deployment:** Optimize models using TensorFlow Lite for deployment on resource-constrained onboard processors typical of spacecraft systems.
- **Streamlit Dashboard:** Build an interactive mission control dashboard for real-time visualization of anomaly scores, RUL predictions, and decision logs.

7. References

9. Saxena, A., et al. "Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation." IEEE International Conference on Prognostics and Health Management, 2008.
10. Liu, F.T., Ting, K.M., and Zhou, Z.H. "Isolation Forest." IEEE International Conference on Data Mining, 2008.
11. Hochreiter, S. and Schmidhuber, J. "Long Short-Term Memory." Neural Computation, 9(8), 1997.
12. Schölkopf, B., et al. "Estimating the Support of a High-Dimensional Distribution." Neural Computation, 13(7), 2001.
13. Zheng, S., et al. "Long Short-Term Memory Network for Remaining Useful Life Estimation." IEEE International Conference on Prognostics and Health Management, 2017.
14. NASA Prognostics Center of Excellence. C-MAPSS Dataset. https://data.nasa.gov/dataset/C-MAPSS-Aircraft-Engine-Simulator-Data/

8. Tools and Technologies

Category	Tools Used
Programming Language	Python 3.10
Data Processing	pandas, NumPy
Machine Learning	scikit-learn (Isolation Forest, One-Class SVM)
Deep Learning	TensorFlow / Keras (Autoencoder, LSTM)
Visualization	Matplotlib, Seaborn
Model Persistence	joblib, Keras model saving
Version Control	Git, GitHub
IDE	VS Code